

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM4538 - IOS İLE MOBİL UYGULAMA GELİŞTİRME II

TASKIO

Taskio Haftalık İlerleme Dokümanı

Elem Yalçinkaya - 22290419

Video linki:

Proje Videoları Oynatma Listesi

1 1. Hafta

Task.io, kullanıcıların görevlerini yönetebildiği ve ekip üyeleri ile birlikte çalışabildiği bir görev yönetimi uygulamasıdır. Kullanıcılar dahil oldukları panolar üzerinden görevlerini To Do, In Progress, Review ve Done gibi durumlara göre sınıflandırabilmekte ve aynı pano üzerindeki diğer kullanıcılarla etkileşim halinde çalışabilmektedir.

Projenin ilk haftasında uygulamanın frontend ve backend tarafında temel iskelet yapısını oluşturdum.

Frontend tarafında React Native kullanıyorum. Tüm frontend kodları src klasörü altında yer almaktadır. Bu klasör içerisinde uygulama için gerekli farklı modülleri ayrı klasörler halinde düzenledim.

Constants klasörü uygulama içerisinde tekrar kullanılacak sabit değerleri içerir. Bu klasörde bulunan api.js dosyasında backend url adresi ve uygulama içerisinde kullanılacak API endpointleri tanımlanmıştır.

AuthContext dosyası ise kullanıcı tokenlarının yönetildiği ve kontrol edildiği yapıdır. Uygulama içerisinde çoğu işlem kullanıcı doğrulaması gerektirdiği için bu dosya içerisinde token kontrolü ve kullanıcı oturum yönetimi gerçekleştirilmektedir.

Navigation klasörü uygulamadaki ekran geçişlerinin ve yönlendirmelerin yönetildiği yerdir. Bu klasörde yer alan AppNavigation yapısı kullanıcının giriş yapıp yapmamasına göre hangi ekranlara yönlendirileceğini belirler. Ayrıca uygulamada kullandığım bottom tab bar navigasyonu da bu klasör içerisinde tanımlanmıştır.

Screens klasörü uygulamanın kullanıcı arayüzü ekranlarını içermektedir. Her ekran farklı bir işlevi temsil ettiği için bu ekranları ayrı klasörler altında düzenledim. Auth klasörü giriş yapma, kayıt olma ve şifremi unuttum gibi kimlik doğrulama işlemlerine ait ekranları barındırmaktadır. Bunun dışında Board, Home, Profile ve Task ekranları da ayrı klasörler altında yer almaktadır.

Frontend tarafında ayrıca Services klasörü bulunmaktadır. Bu klasörde bulunan servis dosyaları frontend ile backend arasındaki HTTP iletişimini sağlar. Uygulama ekranları doğrudan API çağrısı yapmak yerine bu servisler aracılığıyla backend ile iletişim kurmaktadır. Proje ilerledikçe bu servis yapısı genişletilecektir.

Backend tarafında Java Spring Boot kullanıyorum. Backend mimarisini katmanlı bir yapı olacak şekilde tasarladım.

Entity sınıfları veritabanındaki tabloların Java karşılıklarını temsil etmektedir. Her entity sınıfı veritabanında yer alan bir tabloyu modellemektedir. Şu anda veritabanı bağlantısını tamamen kurmadığım için entity içerikleri ilerleyen aşamalarda değişebilir veya genişletilebilir.

DTO yapısını Request ve Response olmak üzere iki farklı şekilde tasarladım. DTO kullanılmasının nedeni entity sınıflarını doğrudan dış dünyaya açmak yerine daha güvenli ve kontrollü bir veri aktarımı sağlamaktır. Request DTO'ları frontend'den gelen veriyi karşılamak için kullanılırken Response DTO'ları backend tarafından oluşturulan verinin frontend'e geri gönderilmesini sağlar.

Controller sınıfları uygulamanın API endpointlerinin tanımlandığı katmandır. Frontend tarafından gönderilen HTTP istekleri controller katmanı tarafından karşılanır ve uygun servis katmanına yönlendirilir.

Repository katmanı veritabanı işlemlerinin gerçekleştirildiği katmandır. Repository sınıfları veritabanı ile doğrudan iletişim kurar ve gerekli sorguların çalıştırılmasını sağlar.

Service katmanı ise uygulamanın iş mantığının yürütüldüğü bölümdür. Servis yapısını Interface ve Implementation olarak ikiye ayırdım. Interface sınıfları yapılacak işlemleri tanımlarken Implementation sınıfları bu işlemlerin nasıl gerçekleştirileceğini belirler.

Resources klasörü içerisinde bulunan application.properties dosyasında ilerleyen aşamalarda veritabanı bağlantı ayarları tanımlanacaktır.

Genel olarak projenin ilk haftasında frontend ve backend tarafında temel mimari ve iskelet yapısını oluşturdum. Sonraki aşamalarda veritabanı tarafına geçerek tabloları Code First yaklaşımı ile oluşturmayı planlıyorum. Projenin ilerleyen haftalarında frontend ve backend geliştirmelerini birlikte ilerleterek uygulamayı daha fonksiyonel hale getirmeyi hedefliyorum.

Ayrıca uygulamada frontend ve backend arasındaki iletişim akışını da kısaca açıklayacak olursam; kullanıcı arayüzünde herhangi bir işlem gerçekleştirdiğinde frontend tarafındaki servis dosyası devreye girer ve backend API'sine bir istek gönderir. Bu istek gönderilmeden önce api.js dosyası aracılığıyla istek header'ına kullanıcı tokeni eklenir.

Daha sonra istek backend tarafındaki Controller katmanına ulaşır. Controller bu isteği yorumlar ve uygun DTO yapısına dönüştürerek Service katmanına iletir. Service katmanında uygulamanın iş mantığı çalıştırılır ve gerekli işlemler gerçekleştirilir. Eğer

veritabanı işlemi gerekiyorsa bu işlemler Repository katmanı aracılığıyla yapılır.

Veritabanından alınan entity verileri doğrudan kullanıcıya gönderilmez. Bunun yerine güvenli bir yapı sağlamak amacıyla DTO Response nesnelerine dönüştürülür. Bu Response DTO daha sonra controller katmanına iletilir ve JSON formatında HTTP response olarak frontend'e gönderilir.

Frontend tarafında servis dosyası bu cevabı alır ve ilgili ekranın state'ini güncelleyerek sonucu kullanıcı arayüzüne yansıtır.

2 2. Hafta

Bu hafta proje kapsamında üç temel değişiklik yapılmıştır. İlk olarak, veritabanı bağlantısı yapılandırılmıştır. Güvenli bir bağlantı oluşturulmuş ve mevcut bağlantı ayarları application.properties dosyası üzerinden yönetilir hale getirilmiştir. Ayrıca, bu bağlantı bilgileri environment değişkenleri aracılığıyla tanımlanarak projenin daha güvenli ve dinamik bir yapıya kavuşması sağlanmıştır. MSSQL veritabanı kullanılarak Code First yaklaşımı benimsenmiş ve bu sayede gerekli tablolar ile entity yapıları otomatik olarak oluşturulmuştur.

İkinci olarak, projeye Expo entegrasyonu gerçekleştirilmiştir. Bu entegrasyon sayesinde, frontend tarafında yapılan değişikliklerin mobil cihaz üzerinde hızlı ve etkin bir şekilde test edilmesi mümkün hale getirilmiştir. Projede başlangıçta birden fazla navigation yapısı bulunmakta iken, yapılan düzenlemeler sonucunda bu yapı tek bir navigation akışı altında birleştirilmiştir. Böylece proje daha modüler, okunabilir ve sürdürülebilir bir hale getirilmiştir. Ana giriş noktası App.tsx dosyası olup, navigation yapısı ve diğer bileşenler Screen ve Service dosyaları altında organize edilmiştir. Bundan sonraki geliştirmelerin Expo tabanlı frontend yapısı üzerinden sürdürülmesi planlanmaktadır.

Üçüncü olarak, environment değişkenleri frontend ve backend tarafında düzenlenmiştir. Frontend tarafında daha önce doğrudan kod içerisinde tanımlanan API URL bilgileri environment değişkenleri aracılığıyla yönetilir hale getirilmiştir. Bu sayede farklı cihaz veya ağ koşullarında manuel müdahale gereksinimi ortadan kaldırılmıştır. Benzer şekilde backend tarafında da environment example dosyası oluşturularak veritabanı bağlantısı ve JWT anahtarları gibi kritik bilgiler güvenli bir şekilde yapılandırılmıştır. Ayrıca .gitignore dosyasında yapılan güncellemeler ile environment dosyaları ve yüksek

boyutlu asset içeriklerinin sürüm kontrol sistemine dahil edilmesi engellenmiştir.

Veritabanı tarafında MSSQL kullanılmakta olup, Code First yaklaşımı sonucunda mevcut durumda sekiz adet temel tablo oluşturulmuştur. Bu tablolar, projenin temel veri yapısını oluşturmaktadır ve proje geliştikçe tablo sayısının artırılması planlanmaktadır. Tablolar arasında gerekli anahtar (key-value) ilişkileri tanımlanmış ve bu ilişkiler bir veritabanı diyagramı ile görselleştirilmiştir. Her bir tabloda kullanıcıdan alınan veriler ve bu verilere ait özellikler sistematik bir şekilde yapılandırılmıştır.

Sonuç olarak, bu hafta yapılan geliştirmeler ile projenin hem backend hem de frontend mimarisi daha güvenli, sürdürülebilir ve ölçeklenebilir bir yapıya kavuşturulmuştur. Projenin ilerleyen aşamalarında bu yapı üzerine yeni özelliklerin eklenmesi planlanmaktadır.

3 3. Hafta

Bu hafta proje kapsamında dört ana başlık altında önemli geliştirmeler yapılmıştır. İlk olarak, kimlik doğrulama (authentication) süreçleri yeniden yapılandırılmıştır. Kayıt (register) işlemi sonrasında kullanıcıyı manuel olarak giriş ekranına yönlendiren yapı kaldırılmış ve bunun yerine otomatik giriş (auto-login) mekanizması geliştirilmiştir. Backend'den dönen token ve kullanıcı bilgileri AsyncStorage üzerine kaydedilerek aynı anda global state (AuthContext) içerisine aktarılmıştır. Böylece kullanıcı kayıt olur olmaz doğrudan uygulama ana ekranına yönlendirilmekte ve kullanıcı deneyimi iyileştirilmektedir. Ayrıca çıkış (logout) işlemi sadeleştirilmiş, gereksiz onay pencereleri kaldırılarak işlem doğrudan gerçekleştirilecek şekilde optimize edilmiştir.

İkinci olarak, frontend tarafında navigation (sayfa geçişleri) ve arayüz yapısında düzenlemeler yapılmıştır. Görev ekleme ekranında oluşan çökme hatasının, ilgili ekranın navigation stack'e eklenmemesinden kaynaklandığı tespit edilmiş ve bu ekran sisteme dahil edilerek sorun giderilmiştir. Bunun yanı sıra, backend ve frontend arasındaki veri formatı uyumsuzluklarını çözmek amacıyla bir normalizasyon katmanı geliştirilmiştir. Backend tarafında TODO ve IN_PROGRESS şeklinde tutulan görev durumları, frontend tarafında kullanıcı dostu biçime To Do ve In Progress olarak çevrilerek Kanban yapısındaki boş kolon problemi ortadan kaldırılmıştır. Ayrıca API isteklerine kullanıcıya ait userId parametresi eklenerek backend ile veri alışverişi tutarlı hale getirilmiştir.

Üçüncü olarak, API bağlantıları ve ağ iletişimi ile ilgili problemler ele alınmıştır. Yerel ağda IP adresinin değişmesi nedeniyle oluşan bağlantı hataları tespit edilmiş ve API adresi environment değişkenleri üzerinden yönetilecek şekilde yeniden düzenlenmiştir. Bu sayede farklı ağ ortamlarında manuel IP güncelleme ihtiyacı ortadan kaldırılmıştır. Aynı zamanda zaman aşımı (timeout) ve hatalı istek (request failed) problemleri çözülerek uygulamanın backend ile daha stabil iletişim kurması sağlanmıştır.

Dördüncü olarak, backend ve veritabanı tarafında önemli sistemsel sorunlar giderilmiştir. `application.properties` dosyası environment değişkenlerini okuyacak şekilde yapılandırılmış ve kritik bilgiler (veritabanı şifresi vb.) daha güvenli hale getirilmiştir.

Bu hafta yapılan geliştirmeler ile projenin kimlik doğrulama altyapısı, kullanıcı deneyimi, API iletişimi ve sistem kararlılığı önemli ölçüde iyileştirilmiştir.

4 4. Hafta

Bu hafta proje kapsamında dört temel değişiklik yapılmıştır. İlk olarak, görev güncelleme sürecinde backend tarafında önemli bir eksiklik giderilmiştir. Daha önce görev güncellenirken `boardId` alanı dikkate alınmadığı için bir görevin bağlı olduğu pano değiştirilememektedir. Bu sorunu çözmek amacıyla `UpdateTaskRequest` veri transfer nesnesine `boardId` alanı eklenmiş ve `TaskServiceImpl` içerisinde ilgili kontrol mekanizması geliştirilmiştir. Böylece gönderilen istekte `boardId` mevcut ise, ilgili pano veritabanından çekilerek görevin pano bilgisi güncellenebilir hale getirilmiştir. Bu düzenleme ile backend tarafında daha esnek ve tam işlevsel bir güncelleme yapısı sağlanmıştır.

İkinci olarak, frontend ve backend arasındaki görev listeleme endpoint uyumsuzluğu giderilmiştir. Önceden frontend tarafında kullanılan endpoint yapısı backend ile eşleşmediği için görevler doğru şekilde çekilememektedir. Bu problem `api.js` dosyasında yapılan düzenleme ile çözülmüş ve endpoint yapısı backend controller ile uyumlu hale getirilmiştir. Ayrıca sabit IP adresi kullanımından kaynaklanan bağlantı problemlerini önlemek amacıyla `Platform.OS` kontrolü eklenmiş, böylece uygulamanın web ortamında localhost, mobil cihazda ise güncel IP adresi üzerinden çalışması sağlanmıştır. Bu değişiklik ile uygulama farklı ağ koşullarında daha stabil ve dinamik hale getirilmiştir.

Üçüncü olarak, uygulamanın navigation yapısı yeniden tasarlanmıştır. Önceki yapıda tek bir stack navigator kullanıldığı için detay sayfalarına geçildiğinde alt menü

kaybolmaktaydı. Bu sorunu çözmek amacıyla nested navigator mimarisi uygulanmış ve her sekme için ayrı stack navigator yapıları oluşturulmuştur. Böylece alt menü her zaman görünür kalırken, detay sayfalarına geçiş ilgili sekmenin kendi stack yapısı içerisinde gerçekleştirilmiştir. Ayrıca AppNavigator içerisinde kullanıcı giriş yapmadığında oluşan boş ekran problemi giderilmiş ve AuthStackNavigator tekrar entegre edilmiştir. Bunun yanında BottomTabNavigator yeniden düzenlenmiş, sekmeler eşit genişlikte hizalanmış ve add butonu daha modern ve uyumlu bir tasarıma kavuşturulmuştur. Bu değişiklikler ile uygulamanın kullanıcı deneyimi ve kod yapısının sürdürülebilirliği önemli ölçüde iyileştirilmiştir.

Dördüncü olarak, görev oluşturma, güncelleme ve silme işlemleri frontend tarafında tam anlamıyla işlevsel hale getirilmiştir. AddTaskScreen bileşeni çift modlu çalışacak şekilde yeniden yapılandırılmış ve tek bir ekran üzerinden hem yeni görev ekleme hem de mevcut görevi düzenleme imkânı sağlanmıştır. TaskDetailScreen içerisine düzenleme ve silme butonları eklenmiş, useFocusEffect kullanılarak kullanıcı geri döndüğünde verinin otomatik olarak güncellenmesi sağlanmıştır. Ayrıca navigasyon sırasında oluşabilecek hataların önüne geçmek için navigate yerine push kullanımı tercih edilmiştir. Bu geliştirmeler sayesinde CRUD işlemleri uçtan uca eksiksiz ve stabil bir şekilde çalışır hale getirilmiştir.

Sonuç olarak, bu hafta yapılan geliştirmeler ile hem backend hem de frontend tarafında veri yönetimi, navigation yapısı ve kullanıcı etkileşimi önemli ölçüde iyileştirilmiştir. Proje daha modüler, sürdürülebilir ve kullanıcı dostu bir yapıya kavuşturulmuş olup, ilerleyen aşamalarda bu yapı üzerine yeni özelliklerin eklenmesi planlanmaktadır.